Applied Machine Learning

(AL-4003)

LABORATORY MANUAL



Lab # 08: Machine Learning with Tree Based Models in Python

Student Name: _____

Roll No: _____

Section: _____

Learning Objectives:

- To learn how to solve machine learning problems by using Decision Trees Classifier
- To learn how to solve machine learning problems by using Decision Trees Regressor

Equipment Required:

- Computer Workstation
- Anaconda (Jupyter Notebook)

Introduction:

Decision Trees

Decision Trees (DTs) are a non-parametric supervised learning method used for classification and regression. The goal is to create a model that predicts the value of a target variable by learning simple decision rules inferred from the data features.

Some advantages of decision trees are:

1. Simple to understand and to interpret. Trees can be visualised.

2. Requires little data preparation. Other techniques often require data normalisation; dummy variables need to be created and blank values to be removed. Note however that this module does not support missing values (older version).

3. The cost of using the tree (i.e., predicting data) is logarithmic in the number of data points used to train the tree.

4. Able to handle both numerical and categorical data. Other techniques are usually specialised in analysing datasets that have only one type of variable.

5. Able to handle multi-output problems.

6. Uses a white box model. If a given situation is observable in a model, the explanation for the condition is easily explained by Boolean logic. By contrast, in a black box model (e.g., in an artificial neural network), results may be more difficult to interpret.

7. Possible to validate a model using statistical tests. That makes it possible to account for the reliability of the model.

8. Performs well even if its assumptions are somewhat violated by the true model from which the data were generated.

The disadvantages of decision trees include:

1. Decision-tree learners can create over-complex trees that do not generalise the data well. This is called overfitting. Mechanisms such as pruning, setting the minimum number of samples required at a leaf node or setting the maximum depth of the tree are necessary to avoid this problem.

2. Decision trees can be unstable because small variations in the data might result in a completely different tree being generated. This problem is mitigated by using decision trees within an ensemble.

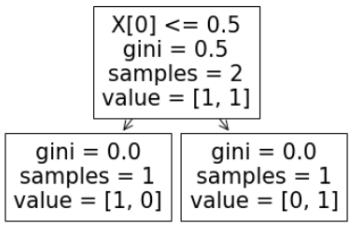
3. The problem of learning an optimal decision tree is known to be NP-complete under several aspects of optimality and even for simple concepts. Consequently, practical decision-tree learning algorithms are based on heuristic algorithms such as the greedy algorithm where locally optimal decisions are made at each node. Such algorithms cannot guarantee to return the globally optimal decision tree. This can be mitigated by training multiple trees in an ensemble learner, where the features and samples are randomly sampled with replacement.

4. There are concepts that are hard to learn because decision trees do not express them easily, such as XOR, parity or multiplexer problems.

5. Decision tree learners create biased trees if some classes dominate. It is therefore recommended to balance the dataset prior to fitting with the decision tree.

In sklearn, ***DecisionTreeClassifier*** is a class capable of performing multi-class classification on a dataset. As with other classifiers, DecisionTreeClassifier takes as input two arrays: an array X, sparse or dense, of size [n_samples, n_features] holding the training samples, and an array Y of integer values, size [n_samples], holding the class labels for the training samples:

```
from sklearn import tree  # importing tree class
X = [[0, 0], [1, 1]]  # making dummy dataset
Y = [0, 1] # making dummy labels
clf = tree.DecisionTreeClassifier() # accessing the classifier
clf = clf.fit(X, Y) # creating a model
clf.predict([[2., 2.]]) # making class prediction
clf.predict_proba([[2., 2.]]) # making class probablity prediction
tree.plot_tree(clf) # displaying the structure of the tree
tree.score(X,y) # check the accuracy of  input X w.r.t y / training accuracy
```



Decision Tree Regressor is a supervised learning algorithm used for predicting continuous numerical values instead of discrete class labels. It works by recursively splitting the dataset into smaller subsets based on input features that minimize prediction error (such as Mean Squared Error).

Each internal node of the tree represents a decision rule based on one feature, and each leaf node represents a predicted numeric value.

Decision Tree Regressors are powerful for non-linear relationships where traditional linear models might fail. They can automatically capture complex patterns without requiring data scaling or transformations.

Parameter	Controls	Effect on Model
`criterion`	Error measure (MSE)	Defines split quality
`splitter`	Split search strategy	"best" vs "random" splits
`max_depth`	Tree depth	Overfitting vs underfitting
`min_samples_split`	Samples to split	Controls node splitting
`min_samples_leaf`	Samples per leaf	Smoothness of prediction
`min_weight_fraction_leaf`	Fraction per leaf	Weighted version
`max_features`	Features to consider	Model randomness
`random_state`	Random seed	Reproducibility
`max_leaf_nodes`	Max leaves	Tree complexity
`min_impurity_decrease`	Min impurity reduction	Prevents unnecessary splits
`ccp_alpha`	Pruning parameter	Simplifies tree
`monotonic_cst`	Monotonic constraints	Keeps direction of relationship

```
from sklearn import tree  # importing tree class

# Creating a small dummy dataset for regression (house size, bedrooms → 
price)
X = [[2104, 3], [1600, 3], [2400, 3], [1416, 2]]   # features: [Size, 
Bedrooms]
y = [399900, 329900, 369000, 232000]               # target: house price

# Accessing the Decision Tree Regressor
reg = tree.DecisionTreeRegressor()

# Training (fitting) the model
reg = reg.fit(X, y)

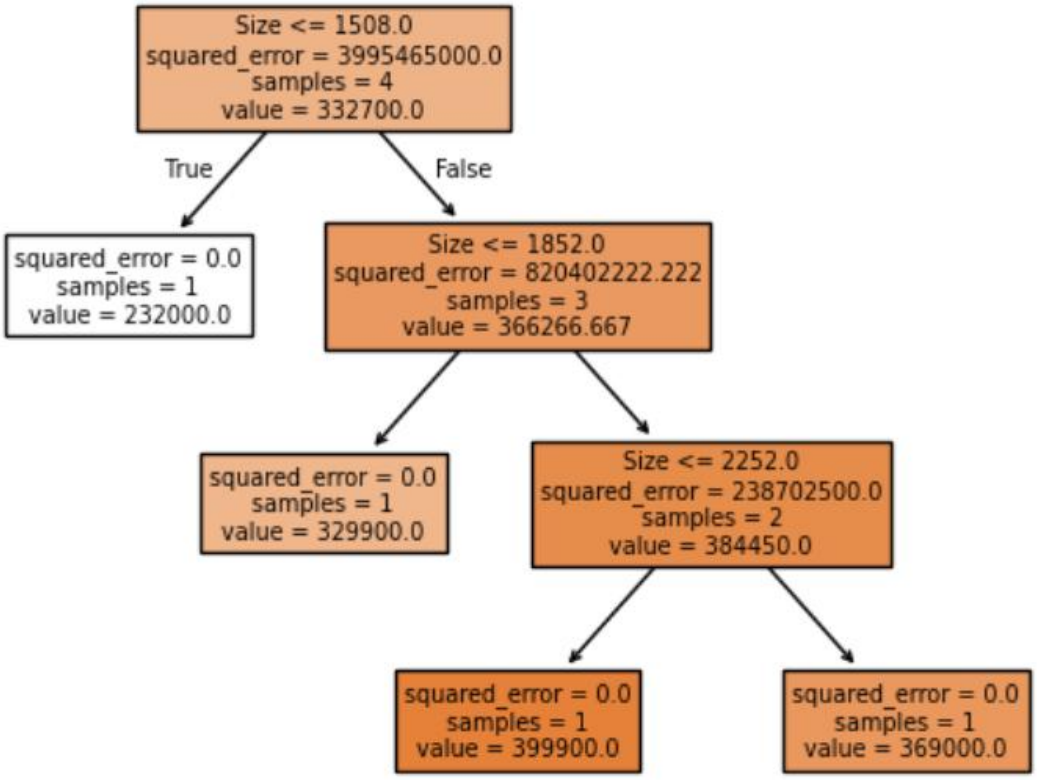
# Making prediction for a new house
print("Predicted Price:", reg.predict([[2000, 3]]))

# Plotting the regression tree structure
tree.plot_tree(reg, filled=True, feature_names=["Size", "Bedrooms"])
```

```
# Evaluate Model

from sklearn.metrics import mean_squared_error, r2_score
y_pred = reg.predict(X)
print("Mean Squared Error:", mean_squared_error(y, y_pred))
# Evaluating model performance (R² Score)
print("R² Score:", r2_score(y, y_pred))
```

```
Predicted Price: [399900.]
Mean Squared Error: 0.0
R² Score: 1.0
```



What is Pruning?

Pruning means cutting or controlling the size of a Decision Tree to prevent it from overfitting (memorizing training data instead of learning general patterns). Just like gardeners trim a tree to help it grow healthier, we prune a decision tree to make it simpler, faster, and more accurate on new data.

Pre-Pruning (Stop Tree Growth Early)

Idea: "Don't let the tree grow too big in the first place." We set limits when building the tree — for example, how deep it can grow, or how many samples are needed to make a split.

```
from sklearn import tree
# Pre-pruning: stop the tree early
reg_pre = tree.DecisionTreeRegressor(
    max_depth=2,          # Limit the tree to 2 levels
    min_samples_leaf=2,   # Each leaf must have at least 2 samples
    random_state=0
)
reg_pre.fit(X, y)

# max_depth=2: Tree stops after 2 levels (no matter what).
# min_samples_leaf=2: Every leaf must have at least 2 data points.
# So, the tree becomes smaller and simpler – it doesn't try to perfectly fit
every data point.
```

Goal of Pre-Pruning

- Prevent the tree from growing too complex.
- Avoid overfitting early.
- Might underfit (too simple) if you stop it too early.
- Pre-pruning controls overfitting, but can cause underfitting if overdone.

Post-Pruning (Trim the Tree After Full Growth)

The tree is trained fully, and then we remove parts that don't add much improvement in accuracy — using a parameter called ccp_alpha (cost-complexity pruning).

```
#Train a full tree first
reg_full = tree.DecisionTreeRegressor(random_state=0)
reg_full.fit(X, y)

#Get possible pruning values (alphas)
path = reg_full.cost_complexity_pruning_path(X, y)
ccp_alphas = path.ccp_alphas

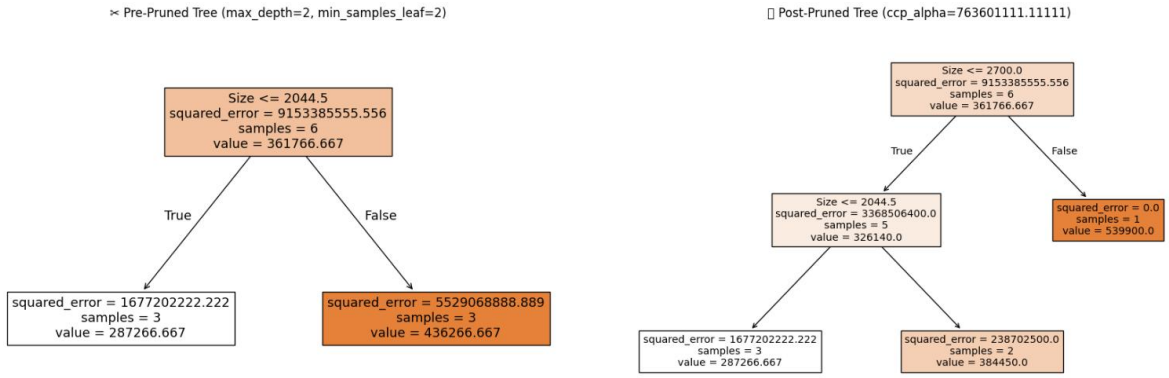
#Choose a pruning strength (alpha)
chosen_alpha = ccp_alphas[len(ccp_alphas)//2]

#Apply post-pruning
reg_post = tree.DecisionTreeRegressor(random_state=0, ccp_alpha=chosen_alpha)
reg_post.fit(X, y)
```

```
# The model first grows a large, complex tree.
# Then, ccp_alpha helps prune (cut) branches that don’t improve the model
much.
# The larger the ccp_alpha, the more aggressively the tree is pruned.
```

Goal of Post-Pruning

- Build full model first, then simplify it.
- Keeps important splits and removes unhelpful ones.
- Slightly more computationally expensive.
- Post-pruning reduces overfitting without much underfitting risk.

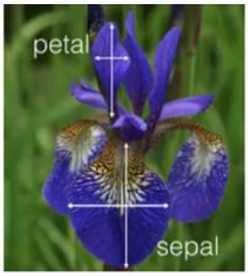


Lab Exercise

Task 1

Iris dataset has 50 samples for each different species of Iris flower (total of 150). For each sample you have sepal length, width and petal length and width and a species name (class/label). In this task you will train the decision tree classifier on Iris dataset. This time you will consider petal length and width as a training dataset (last 2 columns of iris data).

Your task is to build a decision tree classifier model which classifies the new species based on the petal measurements.




```
5.1,3.8,1.9,0.4,Iris-setosa
4.8,3.0,1.4,0.3,Iris-setosa
5.1,3.8,1.6,0.2,Iris-setosa
4.6,3.2,1.4,0.2,Iris-setosa
5.3,3.7,1.5,0.2,Iris-setosa
5.0,3.3,1.4,0.2,Iris-setosa
7.0,3.2,4.7,1.4,Iris-versicolor
6.4,3.2,4.5,1.5,Iris-versicolor
6.9,3.1,4.9,1.5,Iris-versicolor
5.5,2.3,4.0,1.3,Iris-versicolor
6.5,2.8,4.6,1.5,Iris-versicolor
5.7.2.8.4.5.1.3.Iris-versicolor
```

Iris flower: sepal length, sepal width, petal length and width

```
# Import the built-in dataset available in sklearn datasets
from sklearn.datasets import load_iris

# Import Pandas
import pandas as pd

iris = load_iris() # creating iris object to access its functions
# Each observation represents one flower and 4 columns represents 4 measurements

# modify the below code to access only the petal length and width
X=iris.data
y=iris.target

# Convert to DataFrame
df = pd.DataFrame(data=X, columns=iris.feature_names)

# Add target column (species)
df['target'] = iris.target

# Print the first 5 rows
print (_______)

# by using the iris object print the following outputs
# 1. feature_names, 2. target, 3. target_names 4. type of data 5. type target
6. shape of data
# you can use type function to print the type of data and target
```

```
# your code here ------------------------
# train the decision tree classifier (Note: name your model variable 'clf' to
use the plotting function below)
# plot the tree structure

# make class and probability prediction for the following data [[5, 1.5]]
_____________ # making class prediction
_____________ # making class probablity prediction

# plot the decision boundry
from matplotlib.colors import ListedColormap

# To plot pretty figures
%matplotlib inline
import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np
import os

def save_fig(fig_id, tight_layout=True, fig_extension="png", resolution=300):
    path = os.path.join(fig_id + "." + fig_extension)
    print("Saving figure", fig_id)
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)

def plot_decision_boundary(clf, X, y, axes=[0, 7.5, 0, 3], iris=True,
legend=False, plot_training=True):
    x1s = np.linspace(axes[0], axes[1], 100)
    x2s = np.linspace(axes[2], axes[3], 100)
    x1, x2 = np.meshgrid(x1s, x2s)
    X_new = np.c_[x1.ravel(), x2.ravel()]
    y_pred = clf.predict(X_new).reshape(x1.shape)
    custom_cmap = ListedColormap(['#fafab0','#9898ff','#a0faa0'])
    plt.contourf(x1, x2, y_pred, alpha=0.3, cmap=custom_cmap)
    if not iris:
        custom_cmap2 = ListedColormap(['#7d7d58','#4c4c7f','#507d50'])
        plt.contour(x1, x2, y_pred, cmap=custom_cmap2, alpha=0.8)
    if plot_training:
        plt.plot(X[:, 0][y==0], X[:, 1][y==0], "yo", label="Iris setosa")
        plt.plot(X[:, 0][y==1], X[:, 1][y==1], "bs", label="Iris versicolor")
        plt.plot(X[:, 0][y==2], X[:, 1][y==2], "g^", label="Iris virginica")
        plt.axis(axes)
    if iris:
        plt.xlabel("Petal length", fontsize=14)
        plt.ylabel("Petal width", fontsize=14)
    else:
```

```
        plt.xlabel(r"$x_1$", fontsize=18)
        plt.ylabel(r"$x_2$", fontsize=18, rotation=0)
    if legend:
        plt.legend(loc="lower right", fontsize=14)

plt.figure(figsize=(8, 4))
plot_decision_boundary(clf, X, y)
plt.plot([2.45, 2.45], [0, 3], "k-", linewidth=2)
plt.plot([2.45, 7.5], [1.75, 1.75], "k--", linewidth=2)
plt.plot([4.95, 4.95], [0, 1.75], "k:", linewidth=2)
plt.plot([4.85, 4.85], [1.75, 3], "k:", linewidth=2)
plt.text(1.40, 1.0, "Depth=0", fontsize=15)
plt.text(3.2, 1.80, "Depth=1", fontsize=13)
plt.text(4.05, 0.5, "(Depth=2)", fontsize=11)

save_fig("decision_tree_decision_boundaries_plot")
plt.show()

# accuracy of your model
```

Task 2

You are given a dataset (ex1data.txt) that contains housing data with the following columns: Feature Description X_1: Size of the house (in square feet), X_2 : Number of bedrooms, y : Price of the house (in $).

Your task is to train a Decision Tree Regressor on this dataset and predict house prices based on size and number of bedrooms.

```
#Import Required Libraries

import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

#Load and Inspect Dataset

# Load dataset from text file
df = pd.read_csv('ex1data.txt', header=None, names=['Size', 'Bedrooms',
'Price'])
```

```
# Print the first 5 rows
print(_______)

#Split Features and Target

X = df[['Size', 'Bedrooms']]
y = df['Price']

#Train Decision Tree Regressor
# Create model object
model = DecisionTreeRegressor(max_depth= ?, random_state= ?)

# Train the model
#Visualize the Decision Tree
plt.figure(figsize=(12,8))

#Write Code Here
# Make Predictions

# Predict price for a new house 2500 sq.ft house with 3 bedrooms
new_data = _______________
predicted_price = ____________
print("Predicted Price for 2500 sq.ft, 3-bedroom house: $", predicted_price[0])

# --------------------------------
# Post-Pruning (Cost Complexity Pruning)
# Grow full tree first, then prune using ccp_alpha
# --------------------------------
path = ______.cost_complexity_pruning_path(X, y)
ccp_alphas = path.ccp_alphas  # array of possible pruning strengths

# choose a middle alpha value (not too small, not too large)
chosen_alpha = ccp_alphas[len(ccp_alphas)//2]

reg_post = tree.DecisionTreeRegressor(random_state=0, ccp_alpha=chosen_alpha)
reg_post.fit(X, y)

# Evaluate Model

models = {
    "No Pruning Tree": ______,
    "Post-Pruned Tree": ______
}

for name, model in models.items():
    y_pred = model.predict(X)
    print(f"\n--- {name} ---")
```

```
    print("Predicted:", y_pred)
    print("MSE:", mean_squared_error(y, y_pred))
    print("R² Score:", r2_score(y, y_pred))

# ---------------------------------
# Visual Comparison of Tree Structures
# ---------------------------------

plt.figure(figsize=(18, 6))

plt.subplot(121)
tree.plot_tree(reg_pre, filled=True, feature_names=["Size", "Bedrooms"])
plt.title("✂️ Pre-Pruned Tree (max_depth=2, min_samples_leaf=2)")

plt.subplot(122)
tree.plot_tree(reg_post, filled=True, feature_names=["Size", "Bedrooms"])
plt.title(f"🔧 Post-Pruned Tree (ccp_alpha={chosen_alpha:.5f})")

plt.tight_layout()
plt.show()

# ---------------------------------
# Predict a new sample for comparison
# ---------------------------------
new_house = [[2000, 3]]

print("\nPredicted Prices for [2000 sq.ft, 3 bedrooms]:")
for name, model in models.items():
    print(f"{name}: ${model.predict(new_house)[0]:.2f}")
```

AL4003 – Applied Machine Learning Lab						
This lab aims to equip students with various scientific python programming techniques, and develop machine learning systems by using core concepts of machine learning to solve engineering problems.						
CLO	**Statement ↓**	**Exemplary (5)**	**Proficient (4)**	**Developing (3)**	**Beginning (2)**	**Novice (1)**
2	**Design** wide variety of machine learning models and analyze the result accordingly.	Solves problems independently without external help or AI; shows strong logic and explains solutions clearly.	Uses minimal external help or AI as reference; builds own logic and answers questions accurately.	Needs guidance (AI, peers, or resources); adapts provided logic with partial understanding.	Relies heavily on external help or instructor; weak logic and incomplete explanations.	Unable to solve the problem; fully dependent on external solutions with no understanding.
3	**Behaves responsibly** regarding safety for oneself and others.	Independent, follows all rules, submits on time, helps peers conceptually (no code sharing).	Mostly independent, minor guidance, follows rules, timely	Relies partly on peers/AI, sometimes careless with rules	Shows little interest, often late, needs reminders, may share or copy code.	Misconduct, ignores rules, fully dependent on others/AI, plagiarized or incomplete work.

CLO	MARKS OBTAINED
2	
3	

Instructor's Signature: _____